

Game Mode: “Just One More Shift...” (web lab; originally “One Shift”)

Status: **Part 1** reviewed 2026-09-29 (decisions in 1.11). **Part 2** drafted 2026-09-29. **Phases A–E** implemented 2026-09-29 and awaiting a browser test; see the progress log in 2.10. **Part 3** (realistic offer prices from the offer study and Toronto trip data) is a spec, 2026-09-30.

The web lab already has a hidden placeholder tab (`#tab-game`, `docs/lab/components/game-tab.html`, toggled with `g` via `toggle_game` in `js/keyboard-handler.js`). The game will take that tab.

Part 1: Game design

1.1 Premise in one paragraph

You drive for a ridehail platform for one shift in a small simulated city. Your car is one of about a hundred on the map. When the platform’s dispatcher picks you for a trip, the world freezes and an **offer card** pops up: a fixed upfront price, how far away the pickup is, and how long the trip is. You have a few seconds to **Accept** or **Decline**. When the shift ends you see what you actually earned per hour, after expenses and counting every unpaid minute, and how that compares with automated drivers who followed simple rules, and with the rest of the fleet.

The lesson we are after is one drivers learn the hard way: **a good trip is not the same as a good hour**. Whether a \$9 offer is worth taking depends on how long the pickup is, where the trip leaves you, and how soon the next offer will come. Upfront pricing hides the rate card behind a single number.

1.2 What changes from the sketch, and why

Sketch	Adjusted design	Reason
“Periodically” gets an offer	Offers come from the dispatcher itself : when the nearest-vehicle dispatch picks the player’s car, that becomes an offer	Offer frequency then comes out of the market (busy vs slow), which is the core lesson. No separate offer generator to calibrate.
5 seconds to decide	World freezes while the card is up; countdown of 8 s (5 s for the retired Pro); timeout = decline	At about 1 block per second of real time, 5 s would be 5 sim-minutes of driving. Freezing keeps the decision clean and fair.

Sketch	Adjusted design	Reason
Offers above/below a rate card	Every trip gets an upfront driver offer = rate card × multiplier , multiplier drawn from a lognormal fitted to the offer study	Grounded in real data (see 1.5). The comparison with the rate card is never shown on the card , because real drivers don't see it; the debrief reveals it afterwards (revised 2026-09-29).
Earnings at the end	Net earnings per hour of shift : fares minus per-km running costs, divided by <i>all</i> shift time (idle and pickup included)	Accounting for idle time, pickup time and running costs is most of what the game teaches.
Compare to automated drivers	Four named bot drivers with simple, visible strategies, plus a fleet percentile (the other ~100 cars accept everything)	Competitive, and it shows which rules of thumb work in which market.
Simple mode, 1 block 1 minute	Yes: 1 block = 1 minute = 0.5 km (30 km/h). Money is computed in the game layer, not by the sim's Costs & Incomes mode	Keeps the sim in Simple mode. The game adds the money on top.
Non-equilibrating	Yes, fixed fleet (equilibration = NONE)	As sketched.

1.3 A shift, step by step

1. Choose a shift (setup card on the Game tab):

- **Market**: *Slow Tuesday* (oversupplied, long idle), *Normal*, or *Busy Friday* (undersupplied, offers come fast). Only the fleet size relative to demand changes.
- **Offer screen** (replaced *Difficulty*, 2026-10-01; see Part 6): *Rate helper* (\$/km and \$/hr incl. pickup shown on the card, as a third-party app would) or *Platform only* (the platform's card alone). That is the only difference: both have 8 s and no acceptance rule, and neither shows the rate card. Each has its own leaderboard.
- **Shift code** (optional): a seed. The same code gives everyone the

same city, demand stream and your starting point, so friends can compare scores. There is a default “Shift of the day” code derived from the date.

2. **Warm-up** (invisible, ~60 blocks run without rendering) so the market is at steady state when your shift starts.
3. **Drive**. The map is the familiar lab map. Your car is enlarged and outlined, and it keeps its phase colour (P1 blue / P2 amber / P3 green, from the unified palette). A HUD strip shows: shift clock (e.g. “1:42 left”), earnings so far, current \$/hr, acceptance rate, and (dormant acceptance rule only) a warning when you are close to the acceptance limit.
4. **Offer**. The world dims and freezes. The pickup and dropoff are highlighted on the map with a line from your car → pickup → dropoff. The card shows (generic styling, *not* Uber/Lyft branding):

\$11.40	5
o 4 mins (2.0 km) away \$/km	
16 mins (8.0 km) trip	\$1.14
	\$/hr
Drop-off: Downtown	\$34.2
[Decline]	[Accept]

The legs are worded as on a real platform card (“13 mins (10.8 km) away”, “2 hr 1 min (141.7 km) trip”). The boxed column appears with the rate helper only: it stands for the third-party apps some drivers run, which overlay the offer per km and per hour (pickup included) on the platform’s card. Platform only shows the platform’s card alone. Unlike real cards, there’s no map inset, no rider rating, and there is a Decline button (the platforms have only Accept and a close ×). Keys: →/Enter accept, ←/Esc decline. Big tap targets on phone. The card does **not** compare the offer with the rate card. Drivers receiving upfront offers aren’t told this, and the uncertainty is part of what makes the decision hard. They could in principle work it out from the published rate card and the trip’s distance and time, but not in a few seconds. The debrief’s offer log reveals it afterwards.

5. Consequences:

- *Accept*: you drive to the pickup (unpaid), then the trip (paid on drop-off).
- *Decline or timeout*: the trip goes back to the dispatcher and goes to the next-nearest car, never back to you. The rider waits longer (we count that, see the debrief).

6. **Time warp:** while you are on a trip there is nothing to decide, so the sim runs about $3\times$ faster until you are idle again. This keeps a 3-hour shift to about 3–4 minutes of play.
7. **End of shift** at the chosen length (default 3 sim-hours = 180 blocks). A trip in progress is credited pro rata for the distance covered, for the player and the bots alike.
8. **Debrief** (see 1.7).

1.4 Economics (all in the game layer)

The sim stays in Simple mode. Constants below are starting values to calibrate in Part 2.

- **Scale:** 1 block = 1 min = 0.5 km. Trip time and distance are therefore exactly proportional (no traffic). This is a deliberate simplification: on the card, \$/km and \$/min carry the same information, so we show only one derived rate (per minute, including pickup).
- **Driver rate card** (the “old” time + distance card): **\$2.50 + \$0.75/km + \$0.18/min**. For the median offer-study trip (6.9 km, 14 min) that gives \$10.20.
- (*Superseded 2026-09-30 by Part 3: offers now come from a model fitted to the offer study.*) **Driver offer** = rate card $\times m$, $m \sim \text{LogNormal}(\text{median } 0.92, \text{ } 0.30)$, clipped to $[0.45, 2.2]$, rounded to \$0.05. Median offer for a median trip is about \$9.40, which matches the offer-study median fare (\$9.39), and a 10–90% spread of about $0.63\text{--}1.35\times$ matches the observed Lyft fare/fit ratio ($0.65\text{--}1.48$). The price is fixed per trip: if you decline, the next driver sees the same price.
- **Rider price:** not modelled in v1 (see 1.10 v2).
- **Running cost:** \$0.56 per km actually driven, idle cruising and pickups included: the median cost per km driven in a 2024 report to the City of Toronto (backgroundfile-251343.pdf, p. 26; fixed and variable expenses, mostly variable). Until 2026-09-30 it was \$0.30, taken from the lab presets’ `per_km_ops_cost`, which has no documented source.
- **Score** = net earnings over the shift = Σ fares – running costs. Headline: **net \$/hr of shift**.

Sanity check (Normal market, accept everything): $P3 \quad 0.5$ gives about 2.5 trips/hr $\times \sim \$9.40 = \$23/\text{hr}$ gross, minus about \$9/hr running cost (30 km/hr if idle cars keep moving), so about **\$14/hr net**. That is realistic, and low enough that players will want to beat it.

1.5 Grounding in the offer study

From `~/src/uberdriver/offer_study` (18,969 extracted Toronto offer cards, Uber/Lyft/Hopp):

Quantity	5%	25%	median	75%	95%
Fare (\$)	3.68	5.99	9.39	15.61	27.28
Trip km	1.4	3.2	6.9	16.7	32.2
Trip min	5	9	14	22	37
Pickup min	1	3	4	6	10
Fare/km incl. pickup	0.52	0.73	0.98	1.37	2.27
Fare per engaged hour	19.5	24.5	29.5	37.0	53.6

- Trip lengths are strongly right-skewed. The game should use the sim’s **GAMMA** trip-distance distribution rather than **UNIFORM**, so players see a real mix of short trips and long airport-style trips.
- Short trips pay a little less per engaged hour (median ~\$27.5/hr for trips under 15 min vs ~\$33/hr for 25–40 min). A small positive tilt of **m** with trip length could reproduce this. Optional, and low priority.
- The debrief cites the dataset (decision 6, and debrief item 8).

1.6 The bots (and the fleet)

Every non-player car is an automated driver. Most accept everything, which is the sim’s normal behaviour. Four cars are named bots whose rule is shown in the debrief. They receive offers from the same dispatcher and price model and decide instantly:

Bot	Rule	Teaches
Yes-to-Everything	accept all	the baseline. Its score also defines the fleet
Rate-Card Loyalist	accept only if offer rate card	cherry-picking on price alone. This bot works out the rate card exactly, which a human can’t do in the time allowed
Dollar-a-Km	accept if offer $\div$ (pickup km + trip km) \$1.00	the drivers’ folk rule (the study’s mean is \$1.05/km)
Hourly Thinker	accept if offer $\div$ (pickup min + trip min) \$0.55/min (\$33/engaged hr)	a reservation rate on engaged time

Bot names are functional, so we do not have to pick personal names or genders. Each bot’s result is a single run, so it includes luck. The debrief says so (“one shift; luck matters, try another shift code”).

Fleet percentile: “You out-earned 78% of drivers in this shift” is computed over all cars’ net \$/hr using the same accounting.

Market-dependent ranking (the key lesson): in *Busy Friday* the picky bots tend to win, because a declined offer is replaced within a minute or two. In *Slow Tuesday*, Yes-to-Everything tends to win, because each declined offer costs a long idle wait. We should confirm during calibration that the bot ranking actually flips between the markets. If it doesn’t, the market parameters need adjusting until it does.

1.7 End-of-shift debrief

1. **Score card:** net earnings, net \$/hr, and a rank table (you + 4 bots), with the fleet percentile. Personal best per market/difficulty is kept in `localStorage` (a convenience; there is no server leaderboard because the lab is static on GitHub Pages).
2. **Share line**, Wordle-style, copied to the clipboard: `Ridehail shift 2026-09-29 · Busy · Pro - $19.80/hr, beat 83% of drivers, 2nd of 5`. The shift code makes this comparable between friends.
3. **Where your time went:** a single stacked bar of idle / to pickup / with rider (P1/P2/P3 palette colours), with *unpaid* marked over idle + pickup.
4. **Offer log:** one row per offer with price, % vs rate card (labelled as what the card didn’t show), \$/min incl. pickup, decision, and (for declined offers) what happened next (“taken by a car 2 min further away”). Sortable. Rows colour-coded above/below the rate card.
5. **Three auto-generated insights**, chosen from a small rule set, e.g.
 - “You declined 4 offers that paid above the rate card.”
 - “Pickups were 19% of your shift, all unpaid.”
 - “Your best trip paid \$0.92/min; your worst \$0.31/min.”
 - “In this market the average wait between offers was 3.1 min, so each decline cost you about \$1.20 of expected earnings.”
6. **Engaged vs total time:** “Paid-trip time was 52% of your shift. Your earnings per *engaged* hour were \$31.20; per *shift* hour, \$17.40.” A one-line note that some minimum-pay laws (e.g. Ontario’s Digital Platform Workers’ Rights Act) count only engaged time. The exact current rate should be looked up and checked before we publish.
7. **Rider impact:** “Your declines added a total of N minutes of rider waiting.” This is a small nudge about the effect declines have on the system.
8. **Data note:** “Offer prices are drawn to match about 19,000 real Toronto offer cards collected by drivers (the offer study),” with a link or citation.
9. Buttons: **Same shift again** (same code), **New shift**, **Try another market**.

1.8 The platform pushes back (dormant)

Retired with Pro (Part 6): `GameParams.acceptance_rule` still works, and is tested, but no offer screen turns it on.

Pro turns on an **acceptance-rate rule**: if you accept fewer than 4 of your last 10 offers, you are put in a **10-minute timeout** with no offers (the car greys out on the map). The HUD shows acceptance rate with a warning band. This means you cannot simply decline everything to stop the platform setting the terms. The debrief notes that platforms have used acceptance thresholds and tier programmes this way, and that some now label offers “declining won’t affect your acceptance rate” (a badge that appears on 681 cards in the study).

Bots are exempt: their rules are fixed and they would rarely hit the limit. This is noted in the debrief.

1.9 Market settings (original starting points; calibrated values are in 2.10)

Based on the Town preset (city 24 → 12 km square, inhomogeneity 0.5 so the centre is busier), GAMMA trip distances, mean 12 blocks (6 km), `pickup_time` = 1, `idle_vehicles_moving` = 1.0, equilibration NONE, fixed demand:

Market	Vehicles	Demand (req/block)	Target P1	Rough gap between offers when idle
Busy Friday	~95	5	~0.15	~1–2 min
Normal	120	5	~0.30	~3–5 min
Slow Tuesday	~160	5	~0.45	~8–12 min

Inhomogeneity adds strategy: a long trip to the edge pays well but can strand you where offers are rare, and the Drop-off line on the card hints at this.

1.10 Scope

v1 (the playable game): setup card; three markets, two difficulties, shift code; freeze-and-offer card with countdown and keyboard/tap; highlighted player car and offer route; HUD; time warp while busy; four bots + fleet percentile; debrief sections 1–9; personal bests; share line; Pro acceptance rule; phone layout (offer card as a bottom sheet over the maximised map).

v2 candidates (explicitly out of v1): - **Rider price and platform-take reveal**: a second price model, rider rate card × an independent multiplier, so the platform’s cut varies from trip to trip. Deferred until it can be calibrated against figures we have on rider prices and take rates. - **Repositioning**: while idle, tap the map to drive toward a spot (e.g. downtown). This adds real agency but needs a new “idle with destination” behaviour in the sim. - Demand that varies over the shift (rush hour, then a lull), reusing the hour-of-day machinery. - Surge / “offer price rises after declines” as a scenario, framed as a hypothesis. - Per-driver price discrimination as an explicit scenario (“the platform learns

your reservation price”), again framed as a what-if, not a claim. - Rider rating, badges, stacked (forward-dispatch) offers. - A server leaderboard (needs a backend, which the lab doesn’t have).

1.11 Decisions (reviewed 2026-09-29)

1. **The world freezes during offers.** Agreed.
2. **Acceptance-rate penalty in Pro only.** Agreed. Pro itself is low priority, so it goes in the last phase of the plan.
3. **No rider-price / platform-take reveal in v1.** It moves to v2 and needs calibrating against real figures first.
4. **Repositioning deferred to v2.** Agreed.
5. **Shift length 3 sim-hours** (180 blocks). Agreed.
6. **Cite the offer-study dataset** in the debrief. Agreed.

Part 2: Implementation plan

2.1 Architecture in one picture

Game tab (main thread)		webworker.js	Python (Pyodide)
game-tab.js	Play(gameSimSettings)	init_game(settings)	worker.py GameSimulation
setup card			RideHailSimulation
HUD		getNextFrame loop	Dispatch.on
offer card	frame {vehicles,trips,	(paused while an offer	ridehail/game.py
debrief	game:{...,offer}}	is pending)	GameController
	GameDecision(accept)	resolve_offer()	(prices, bots, ledger)
	GetGameResults	game_results()	offer log, results)
modules/map.js	plotMap(results) + game overlay	(player ring, offer route)	

Principles:

- **All game logic is in Python, in a new core module ridehail/game.py.** That covers the price model, bot rules, the per-vehicle ledger, the offer log, the acceptance rule and the end-of-shift results. It ships in the wheel, is unit-tested with pytest, and can be driven headless by a calibration script. JS only renders it and forwards decisions.
- **The core simulation changes in one place.** An optional `offer_filter` hook in `Dispatch`. When the hook is `None` (every non-game run), `dispatch` behaves byte-for-byte as it does now.
- **The world freezes in the worker, not in the renderer.** When an offer is pending, the worker does not schedule the next frame. The main thread shows the card and sends a decision, which resumes the loop. This reuses the existing backpressure and `activeRunId` machinery unchanged.
- **The game gets its own settings object** (`appState.gameSimSettings`, name `"gameSimSettings"`). It shares the single worker loop with the other

tabs, as What If does, and entering or leaving the Game tab resets it.

2.2 Core simulation change: the offer hook (ridehail/dispatch.py)

Refactor each single-trip dispatch function into **find** and **commit** steps, without changing behaviour:

- `_find_vehicle_sparse(trip, city, candidates)` and `_find_vehicle_dense(trip, city, grid, candidate_set, vehicles)` return `(vehicle, dispatch_distance)` or `None`, and change nothing.
- `commit_dispatch(trip, vehicle)` (public, static) does what the tail of each function does now: `trip.update_phase(WAITING)`, `vehicle.update_phase(trip=trip)`. The caller removes the vehicle from the candidate structures.
- `Dispatch.__init__` gains `self.offer_filter = None`.
- With `offer_filter` is `None`, `_dispatch_vehicles_default` runs the current code path exactly (find → commit), including the random tie-break order, so seeded runs are unchanged.
- With a filter set, each trip loops as follows:

```
excluded = set()
loop:
    found = find(trip, candidates - excluded)      # skip excluded vehicles
    if not found: break                            # trip stays UNASSIGNED
    decision = offer_filter(trip, vehicle, dispatch_distance)
    ACCEPT → commit, remove vehicle from pool, break
    DECLINE → excluded.add(vehicle); continue      # next-nearest, same block
    DEFER → remove vehicle from pool this block; break # player offer held
```

For the dense search, excluded vehicles are skipped with the same set-membership test that already exists (`vehicle not in dispatchable_vehicles_set`). The check becomes `... or vehicle in excluded`.

- Only `DispatchMethod.DEFAULT` supports the hook. The game always uses `DEFAULT`, and the hook raises on other methods if it is set.
- Tests: extend `test/test_dispatch_performance.py` or add `test/test_dispatch_offer_filter.py`:
 1. With a fixed seed and `offer_filter=None`, trip→vehicle assignments match the current implementation over N blocks, for both sparse and dense searches. Record them from the pre-refactor code as a fixture first.
 2. `DECLINE` gives the trip to the next-nearest vehicle in the same block.
 3. `DEFER` leaves the trip `UNASSIGNED` and the vehicle in `P1`.
 4. A vehicle that declined a trip is never offered it again (the `declined_by` set lives on the trip; see 2.3).

2.3 ridehail/game.py

```
class OfferDecision(Enum): ACCEPT, DECLINE, DEFER

@dataclass
class GameParams:          # all calibratable constants from Part 1
    km_per_block = 0.5; min_per_block = 1.0
    rate_base = 2.50; rate_per_km = 0.75; rate_per_min = 0.18
    mult_median = 0.92; mult_sigma = 0.30; mult_clip = (0.45, 2.2)
    ops_cost_per_km = 0.30
    shift_blocks = 180; warmup_blocks = 60
    offer_seconds = 8          # Rookie; 5 for Pro (JS uses it)
    acceptance_rule = None | (min_accepts=4, window=10, timeout_blocks=10)

MARKETS = {"slow": {...}, "normal": {...}, "busy": {...}} # 1.9 table
BOT_RULES = {"yes": ..., "loyalist": ..., "per_km": ..., "hourly": ...}

class GameController:
    def __init__(self, sim, params, seed)
        # picks player + 4 bot vehicle indexes with its own random.Random(seed)
        # installs self._offer_filter on sim._dispatcher.offer_filter
    def warm_up()
        # runs warmup_blocks with the filter as
        # accept-all, ledger off, then resets
        # sim.block_index bookkeeping for the shift
    def before_block()
        # applies a resolved player decision
    def after_block(block)
        # ledger update: see below
    def pending_offer() -> dict|None # for the frame payload
    def resolve_offer(accept: bool) # commit, or mark declined; recompute
        # the player's direction if accepted
    def frame_payload() -> dict      # HUD numbers + player/bot indexes + offer
    def results() -> dict           # the debrief (Part 1 §1.7)
```

Details:

- **Prices** are drawn lazily the first time a trip is offered to anyone, from the controller's own `random.Random`. They are stored in `self.prices[trip_id] = (rate_card, offer)`; trip ids are permanent dict keys in `sim.trips`. A separate RNG means the price draws don't disturb the sim's global `random` stream more than the player's own decisions do.
- **The offer filter** is the hot path, since it runs for every dispatch. It returns `ACCEPT` immediately for any vehicle that is not the player or a bot. For bots it applies `BOT_RULES`. For the player it returns `DEFER` and records `self.pending = {trip_id, vehicle_index, pickup_blocks, ...}`, unless the player is in a Pro timeout, in which case it returns `DECLINE` silently. Declines are logged per trip (`trip.declined_by`, set lazily with `getattr`) so the same vehicle is never offered that trip again.

- **Resolving an offer** happens at a block boundary. On accept: `Dispatch.commit_dispatch(trip, player)` then `player.update_direction()`, because the direction was chosen at the end of the block while the car was still P1. On decline or timeout: add to `declined_by`, log it, and the trip is re-dispatched normally in the next block. The log records what happened next (“taken by a car N min further away”) when the trip is next dispatched.
- **Ledger (after_block)**, per vehicle, vectorised simply over the ~120 cars:
 - km: +0.5 if the location changed since the last block. Wrap-around counts as a move.
 - minutes by phase: +1 in P1/P2/P3.
 - earnings: when a vehicle’s tracked trip id reaches `COMPLETED`, credit `offer` (or `rate_card` for non-bot fleet cars, drawn the same way so fleet earnings are comparable).
 - offers seen / accepted, for the player and bots.
 - To see which trip a vehicle was on, snapshot `vehicle.trip_index` before the block. Completion clears it in `Vehicle.update_phase`.
- **End of shift / pro rata**: in `results()`, a trip in progress (P3) is credited `offer × blocks ridden / trip.distance`. For that, the ledger records the block at which each P3 trip started.
- **Results**: player summary, bot rows, fleet net-\$/hr list (for the percentile), time split, offer log, and insight inputs (declined above-rate-card count, unpaid pickup share, best and worst \$/min, mean gap between offers). The JS side picks and phrases the insights.
- **Acceptance rule** (Pro, last phase): a rolling window of the player’s decisions. Crossing the threshold sets `timeout_until = block + 10`.

Tests (`test/test_game.py`, plain pytest, no Textual): - The price model: median and quantiles of `offer/rate_card` over 10k draws fall within tolerance of the targets, and clipping works. - Each bot rule accepts and declines the right synthetic offers. - The ledger: a scripted trip credits exactly once, km and minutes add up to the number of blocks, and pro rata is correct at the shift end. - End to end: a seeded headless shift with a scripted player (accept all) runs 180 blocks, and results are internally consistent (Σ phase minutes = 180 per car). - Offer determinism: the same seed and the same decision sequence give the same offer sequence.

2.4 Calibration script (before any UI): `utils/game_calibrate.py`

This runs headless shifts: 20 seeds $\times$ 3 markets, with the player seat played by each bot rule in turn plus accept-all. It reports, per market: - P1/P2/P3 fractions, and the mean and 90th-percentile gap between offers for an idle car. - Net \$/hr per strategy (mean $\pm$ sd over seeds) and each bot’s acceptance rate. - **The key check (\$1.6)**: whether the strategy ranking flips between *Busy* and *Slow*.

Tune `MARKETS` (vehicle count, mostly) and, if needed, the bot thresholds until

the flip is clear and accept-all nets roughly \$12–16/hr in Normal. Record the final numbers in this document. This is Python-only, so we can run it directly (no Textual, no browser).

2.5 Worker bridge

`docs/lab/worker.py` - `init_game(settings)`: builds the `RideHailConfig` from the market preset (Town geometry, GAMMA trips, `equilibration=None`, `use_city_scale=False`, `idle_vehicles_moving=1.0`, `pickup_time=1`, seed from the shift code; seed must be non-zero, since `simulation.py` only seeds if it's truthy). Then it creates a `GameSimulation(Simulation)` and runs `warm_up()`. It sets the global `sim` like `init_simulation` does. - `GameSimulation` subclasses the existing `Simulation` so it inherits `next_frame_map()` and its midpoint logic. It overrides `_get_block_results` to call `controller.before_block()` / `after_block()` around `next_block`, and attaches `results["game"] = controller.frame_payload()`. - **Offer timing and interpolation** (this is the delicate part; see the memory note on care with map interpolation): a block runs on an *odd* frame, and its real positions are shown on the next *even* frame. So: - the offer goes into `pending_results["game"]` (the even frame), not the midpoint frame. The card appears when the car visibly arrives at the intersection where it was dispatched. - `resolve_offer(accept)` must also update `self.prev_directions[player]` when an accept changes the player's direction. Otherwise the next midpoint frame shows the car facing its old random direction. - When interpolation is off (`city > 32`), the offer rides on the single frame. Game markets are city 24, so interpolation is always on, but both paths should be handled. - `resolve_offer(accept)` and `game_results()` are thin wrappers that return plain dicts.

`docs/lab/webworker.js` - New `SimulationActions`: `GameDecision`, `GetGameResults` (in `js/constants.js`). - In `getNextFrame`, after conversion: if `results.game?.offer` is set, do **not** arm `pendingFrameSettings`. Keep them in a new `offerHeldSettings/offerHeldRunId` pair. The frame is posted and the loop stops. - On `GameDecision`: call `workerPackage.sim.resolve_offer(accept)`, then, if `offerHeldRunId === activeRunId`, move the held settings into `pendingFrameSettings` and call `scheduleNextFrame()`. A stale decision (the tab was reset in the meantime) is ignored. - **Time warp**: when `results.game.player_phase` is P2 or P3, the wait in `scheduleNextFrame` uses `animationDelay / 3`. This is done worker-side, so there is no round-trip lag. - **Frame limit**: game settings carry `timeBlocks = 180`, and the existing `frameLimit` stops the loop at the shift end. The final frame carries `game.shift_over = true`. - On `GetGameResults`: post `{action: "gameResults", results}`, mirroring `GetResults`.

2.6 Main thread

New files: - docs/lab/game-tab.js: a `GameTab` class, following the `WhatIfTab` pattern (constructed in `app.js`, `setupEventHandlers()`, `resetUIAndSimulation()`). Its state machine is `setup` → `warming` → `driving` → `offer` → `debrief`. - docs/lab/components/game-tab.html: replaces the placeholder with the setup card, map canvas, HUD strip, offer-card <dialog>, and debrief section. - docs/lab/modules/game-offer.js: renders the offer card and runs the countdown (a `requestAnimationFrame` ring). Timeout sends a decline. - docs/lab/modules/game-debrief.js: score table, time-split bar, offer log, insights, share line, and personal bests in `localStorage` (wrapped in `try/catch`). - Styles in `style.css` under a `/* Game */` section, using the palette constants and legible sizes (16px, per the font-size preference).

Changes to existing files: - `app.js`: construct `GameTab`, add case `"tab-game"`: `app.gameTab.resetUIAndSimulation()` to the tab switch, and pause or reset the game when leaving its tab. The existing `window.chart.destroy()` on tab switch already frees the map. - `js/app-state.js`: add `gameSimSettings`. - `js/message-handler.js`: route frames whose `name === "gameSimSettings"` to `plotMap(results)` and then `app.gameTab.onFrame(results)` (HUD, offer card, shift end). Route `action === "gameResults"` to the debrief. - `modules/map.js`: a small, optional **game overlay**, active only when `eventData.get("game")` exists: - player car: the same icon at 1.6× radius with a dark outline ring (an extra dataset, index 2, drawn above the vehicles; this keeps datasets 0/1 and their caches untouched). - while an offer is shown: a dashed polyline player → pickup → dropoff (dataset 3, `showLine: true`), plus a dimming layer over the rest of the map, done with a CSS class on the canvas wrapper rather than inside `Chart.js`. - bots: optionally a small letter badge. This can come later. - `initMap` only needs the extra empty datasets when the game initialises it (an `options.game` flag). - `js/keyboard-handler.js`: while the Game tab is active, the global shortcuts (space = pause, etc.) are suspended except `g`. While the offer card is up, `↵/Enter` accept and `⏏/Esc` decline. The `d/a` letter keys are avoided because `d` is already a global mapping. Pause is allowed between offers but not during one. - `index.html`: remove `tab-hidden` from `#tab-game` once `v1` is ready. Until then the tab stays behind the `g` toggle. - Phone (`js/phone.js`, `body.is-phone`): the offer card becomes a bottom sheet with two half-width buttons at least 56px tall, and the HUD collapses to clock + earnings.

Shift code: the default is `YYYY-MM-DD` (today). `Seed = fnv1a(code + market) | 1`, so it is never zero. The code is shown in the setup card and in the share line, and it can be edited. `?game=CODE&market=busy` in the URL deep-links straight into the Game tab.

2.7 Phasing

Each phase ends in something the user can check.

Phase	Scope	Verified by
A. Core	2.2 dispatch hook + refactor; 2.3 <code>game.py</code> ; tests	<code>pytest test/</code> ; seeded dispatch regression unchanged
B. Calibration	2.4 script; tune <code>MARKETS</code> and bots; write the numbers into this doc	the ranking flip between Busy and Slow
C. Bridge	2.5 <code>worker.py</code> + <code>webworker.js</code> ; <code>./build.sh</code>	the user plays a shift in the browser with the game driven from the console (a temporary debug hook: <code>w.postMessage({action:"gameDecision",accept:tr</code>
D. Playable	2.6 tab, setup card, map overlay, HUD, offer card, keyboard, time warp	the user plays a Rookie shift end to end
E. Debrief	results, time split, offer log, insights, citation, share line, personal bests	the user reviews the debrief across all three markets
F. Polish	phone layout, Pro difficulty + acceptance rule, URL deep link, unhide the tab	the user tests on a phone

Browser checks are manual by the user (Apache serves `docs/lab`; we have no browser access). Every phase that touches Python needs `./build.sh` so the wheel in `docs/lab/dist/` is current.

2.8 Risks and mitigations

- **Dispatch regression.** The refactor touches the hottest code path. The seeded-assignment fixture (2.2 test 1) and the existing `test_dispatch_performance.py` guard it. The `None` filter path adds only one attribute check per block.
- **Interpolation and direction glitches when resuming after an offer.** Handled in 2.5 (offer on the even frame, `prev_directions` patched). The change is kept isolated so it's easy to bisect, as the memory note asks.
- **Shared worker loop races.** The game uses a distinct settings name and the existing `activeRunId` claim. A decision arriving after a reset is dropped by the run-id check.
- **Offer droughts in the Slow market,** where the game gets boring. Time warp also applies while idle for more than N blocks with no offer (optional, decided after calibration), and calibration caps the 90th percentile offer gap.

- **Bot results are noisy from a single shift.** The debrief says so, and the fleet percentile is the steadier measure.
- **Scope creep from v2 ideas.** v2 items stay in §1.10 until v1 has shipped.

2.9 Files at a glance

New: `ridehail/game.py`, `test/test_game.py`, `test/test_dispatch_offer_filter.py`, `utils/game_calibrate.py`, `docs/lab/game-tab.js`, `docs/lab/modules/game-offer.js`, `docs/lab/modules/game-debrief.js`.

Changed: `ridehail/dispatch.py`, `docs/lab/worker.py`, `docs/lab/webworker.js`, `docs/lab/js/constants.js`, `docs/lab/app.js`, `docs/lab/js/app-state.js`, `docs/lab/js/message-handler.js`, `docs/lab/modules/map.js`, `docs/lab/js/keyboard-handler.js`, `docs/lab/js/phone.js`, `docs/lab/components/game-tab.html`, `docs/lab/style.css`, `docs/lab/index.html` (unhide, phase F).

2.10 Progress log

2026-09-29: Phase A (core) done. - `ridehail/dispatch.py`: `OfferDecision` enum, `Dispatch.offer_filter` hook, single-trip dispatch split into `_find_vehicle_sparse/_dense` (pure) and `commit_dispatch` (public). Only the `DEFAULT` method supports the hook. - `test/test_dispatch_offer_filter.py`: seeded full-state digests recorded from the pre-refactor code for three scenarios (dense Town geometry, dense, sparse with backlog). They still match, so dispatch is unchanged when no filter is installed. Plus hook semantics (accept-all equivalence, decline $\rightarrow$ next-nearest, decline-all, defer) on both search paths. - `ridehail/game.py`: `GameParams`, `MARKETS`, `DIFFICULTIES`, four `BOTS`, `Ledger`, `GameController`, `create_game()`, `shift_seed()`. A whole shift runs headless in ~ 0.3 s. `test/test_game.py` has 13 tests. - The full suite has 5 failures that also fail on the untouched `HEAD`. They are not caused by this work: `test_regression.py` (`city.config` takes about 9 minutes against a 5-minute timeout; the other configs' expected results predate newer metrics) and `test_web_animation_quick.py` (uses the removed `config.animate`).

2026-09-29: Phase B (calibration) done. `utils/game_calibrate.py`, 80 seeds per market:

Market	Cars	P1	Idle min / offer	Accept- all	Hourly Thinker	Dollar- a-Km	Rate- Card Loyal- ist
Busy Friday	80	0.00	~ 0	\$14.0	\$28.0	\$25.7	\$18.1
Normal	96	0.20	4.0	\$14.8	\$16.8	\$16.3	\$16.9
Slow Tues- day	130	0.48	12.1	\$9.8	\$8.6	\$9.1	\$5.9

(net \$/hr; standard errors \$0.3–0.7). The ranking flips in steps: being picky roughly doubles earnings when Busy, helps a little in Normal, and costs money when Slow. Shared market: city 24, demand 5/block, mean trip 12 blocks (GAMMA), inhomogeneity 0.5.

What calibration changed in the design: 1. **Riders cancel after 10 minutes unassigned** (`GameParams.max_wait_minutes`). An undersupplied market otherwise has a backlog that grows without limit: below ~92 cars, 70–130 riders were still stranded at the end of the shift. This needed a small core change: `RideHailSimulation.max_wait_time` (default `None`, so nothing changes) passed to the existing `_cancel_requests()` stub. With it, Busy holds ~20 waiting riders and about 15% of requests are abandoned. The player is never offered a rider who is about to give up. 2. **Fair comparison: each driver is measured from their first idle moment.** The player and bots are logged off during warm-up, so they start idle. A fleet car on a trip when the shift starts joins when that trip ends, and the pre-shift trip doesn't count. Net \$/hr is computed over each car's own window; the fleet group leaves out cars with windows under 60 minutes. Without this, accept-all trailed the fleet by ~\$2/hr. With it, accept-all, Yes-to-Everything and the fleet mean agree to within \$0.3/hr in every market. 3. **Earnings accrue per block with the rider aboard** ($\text{offer} \div \text{trip blocks}$, with the remainder at drop-off). This replaces the planned pro-rata at shift end, and a trip in progress when the shift ends is handled the same way for everyone. The ledger keeps its own reference to each paying trip, because the sim's garbage collection can drop a completed trip in the very block it completes. This was a real bug, caught by a test.

Observations for the UI phases: - **Luck.** A single shift's net \$/hr has sd \$3 (Busy) to \$5 (Slow). In Busy, strategy dominates luck. In Normal and Slow, one shift is mostly luck. The debrief should say so, and the shift code (same city, same demand) matters for comparing with friends. - **Decisions per shift.** An accept-all player sees only ~7–10 offers per 3-hour shift, since each trip plus its pickup takes ~20 minutes. A picky player in Busy sees 50+ offers. Time warp should apply while idle in Slow as well as while on a trip, so the waits between offers stay short in real time. To be tuned in Phase D. - **Pickups are long when Busy** ($P2 = 0.5$, ~8 min per pickup). This is realistic, and it gives the pickup line on the card real weight.

2026-09-29: Phases C (bridge), D (playable) and E (debrief) implemented, not yet browser-tested. C, D and E were done together, so the first browser test is the real game rather than a console-driven shift. - `docs/lab/worker.py`: `init_game()` and `GameSimulation(Simulation)`. Wrapper state setup was factored out of `Simulation.__init__` into `_init_frame_state()`. An offer and `shift_over` travel only on real-block frames, and interpolated frames repeat the previous real block's payload. Accepting an offer updates `prev_directions[player]`. Checked headless with a fake Pyodide proxy: no offer ever lands on an odd frame, the card's car position equals the drawn car, and the shift ends on frame 358. The Experiment

path was smoke-tested after the refactor. - docs/lab/webworker.js: game settings flag $\rightarrow$ `init_game`; `offerHeldSettings/RunId` (the offer freeze); `GameDecision` resumes only the run that is holding; `GetGameResults` $\rightarrow$ `{action: "gameResults"}`; time warp via `frameDelayFactor`, which scales both the wait and the frame's `animationDelay` so map glides still finish on time. - docs/lab/modules/game-map-overlay.js: a Chart.js plugin, registered in `initMap` and inert unless `setGameOverlay()` has set state (`initMap` clears it). It draws the player's ring at the car's *animated* position, the offer route (unpaid leg amber, paid leg green, torus-aware) over a dimming wash, and a guide line to the current pickup or drop-off. - docs/lab/game-tab.js (`GameTab`), modules/game-offer.js (card, countdown, $\rightarrow$ / Enter / $\leftarrow$ / Esc), modules/game-debrief.js (score, rank table, fleet percentile, time split, insights, offer log, share line, personal best in `localStorage`, data citation), components/game-tab.html, and a `/* Game tab */` section at the end of `style.css`. - Wiring: `app.js` (construct; leaving the tab stops a game; `gameSimSettings` counter no-op), `js/message-handler.js` (game frames $\rightarrow$ `onFrame` outside the render try/catch; `gameResults` $\rightarrow$ `debrief`; `_simSettingsFor`), `js/keyboard-handler.js` (global shortcuts suspended on the Game tab except `g`, because `space` would otherwise start the Experiment sim on the shared worker loop). - Pacing constants (game-tab.js): 400 ms per frame, time warp $0.3\times$ while busy and $0.75\times$ while idle. Tune after playing. - The debrief renderer was run under Node with real Python results: no `undefined/NaN`. - ./build.sh run; the version moved to 2026.9.29.0.

2026-09-29: Released. After the user's browser test, the tab was unhidden (`tab-hidden` removed from `#tab-game` in `index.html`); `g` still toggles it. Two refinements from that test: the `g` toggle had been broken since commit 84deb61 (the mapping existed only in the generated `keyboard-mappings.json`; `ridehail/keyboard_mappings.py` is now the full source again, and browser and terminal mappings were checked against before), and the offer card no longer compares the offer with the rate card (see 1.3). `build.sh` now also strips the Game tab from the PyPI package's copy of the lab (`ridehail/lab/`, "Experiment + What If only"), and `GameTab.setupEventHandlers()` does nothing when the markup is absent.

2026-09-29: Links to tabs (the Phase F deep link, generalised). Every tab now has a URL hash (`#what-if`, `#read`, `#toronto`, `#game`; Experiment, the default, has none). `app.js` keeps the hash in step with the tab being viewed (`history.replaceState`, so there's no history clutter and no `hashchange` loop), opens the linked tab on load (showing it even if it's hidden), and follows later hash changes. The Game tab accepts `#game?market=busy&code=friday&difficulty=pro` to pre-select its setup screen. It does not auto-start, so a first-time visitor still sees the setup and "How it works". Starting a shift writes that link into the address bar, and the debrief's share line ends with it ("Play the same shift: ..."). Phones: a link is the only way into the Game tab (no tab bar). While it's active the Experiment's phone chrome (hint, speed hint, sheet, action bar) is hidden

and the panel scrolls, and the setup screen has a phone-only “Explore the full simulation in the Lab” link (`#experiment`). The phone autoplay demo now runs only when the Experiment tab is showing; otherwise a `#game` link would start it unseen, holding the worker loop.

2026-09-29: Downtown shading (all maps). When inhomogeneity > 0 , every lab map (Experiment and Game) shades the city core in `MAP_CORE` (first `#d5dce6`, a slightly darker grey; the user then chose steel blue `#7facca` to match the lab’s primary button colour). The game legend’s downtown swatch takes its fill from `MAP_CORE` at runtime, so it can’t drift from the map. `mapCorePlugin` in `modules/map.js` runs after the land background and before the road grid, and follows the live inhomogeneity value frame by frame. Its bounds use the simulation’s own arithmetic (`CITY_CORE_FRACTION` mirrors `City.TWO_ZONE_LENGTH`). This was cross-checked for city sizes 4–200, and it matches the offer card’s “Downtown” label. The game sidebar legend has a “Downtown” entry. Also fixed: a phone had failed to load the engine because it used a stale cached `manifest.json` that named a wheel `deploy.sh --delete` had removed. `webworker.js` now revalidates `manifest.json` and `worker.py` (cache: “no-cache”).

2026-09-29: Pro hidden; footnote reworded. The user decided against a visible difficulty choice: on a real platform every driver gets the same time to decide. The Difficulty fieldset on the setup screen is `hidden`, so every shift is “rookie” (8 s, pay per minute shown). Pro (5 s, no pay per minute, acceptance-rate timeouts) is kept intact under the hood: `ridehail/game.py DIFFICULTIES`, the JS path, and a hidden link switch `#game?...&difficulty=pro`. Share links carry `difficulty` only when it isn’t the default, and the debrief names the difficulty only for Pro. The price footnote now says the rate card and the spread were “calibrated with the aid of” the offer study. That’s accurate: only the median fare for the median trip and the 10–90% spread of the Lyft cards’ fare/fit ratio were used (see 1.4, 1.5). Not reproduced: the study’s short-trip discount, the long-trip tail beyond the 12 km city, platform differences, surge/bonus, and the empirical shape of the spread. Fitting those (a robust rate-card regression, and sampling the empirical fare/rate-card ratios by trip-length band) is a possible next step, and it would need re-calibration of the markets.

2026-09-29: No map on phones. At phone width (the lab’s phone tier, `body.is-phone`, 600px) the map was too small to add much, so on phones the Game tab hides the map and its legend. The status pane (clock, earnings, status line) comes first, and the offer card follows it in the page flow (not a fixed bottom sheet), scrolled into view when it appears. `message-handler.js` skips `plotMap` for game frames in the phone tier. The simulation and game logic are unchanged. Tablets (601–800px) keep the map with the bottom-sheet offer card.

2026-09-30: Leaderboard (option B: named top scores, plus consistency checks). `tomslee.net` runs PHP 8.1.34 with `pdo_sqlite`. - `docs/lab/api/leaderboard.php`: GET a board (`?code=&market=[&difficulty=]`

→ {count, top}), POST a score ({code, market, difficulty, name, version, player} → {count, top, rank, best, name}). A board is shift code × market × difficulty. Each name's best counts; names are case-insensitive, with no sign-in, so any name can be used by anyone. Standings are computed in PHP from the sorted rows, not with SQL grouping, which has version-dependent rules; the first SQL version got ranks wrong because PDO binds numbers as text. - Checks: names 1–20 characters (letters, digits, space, . _ ' -), a built-in blocked-word list plus an optional **blocked-words.txt**; a full, consistent 180-minute shift (minutes sum, km 90, costs = km × \$0.30, net = earnings − costs, net/hr = net ÷ 3, accepts offers, trips accepts, earnings \$300). A carefully faked but consistent result is still accepted: that is the limit of option C, and replay verification (option D) would close it. 30 submissions per hour per address; only a salted address hash is stored, and it is erased after a day. - Storage: SQLite in **RIDEHAIL_DATA_DIR**, else **ridehail-data** three levels above the script's real path (on the host `/home/tomslee/domains/tomslee.net/ridehail-data`, because `~/public_html` is a link to `~/domains/tomslee.net/public_html`; outside `public_html`, so it's never served and never touched by `deploy.sh`'s `rsync --delete`). Created by PHP on first use. - Moderation: put a long random **admin-token** file in the data directory, then GET `...?admin=TOKEN&code=...&market=...` lists entries with ids and POST `{action:"delete", token, id}` deletes one. - Client: **modules/game-leaderboard.js**, a section in the debrief after “How you compare”. It shows the top 10, the count and a name form (name remembered in `localStorage`); ended-early shifts can't be submitted. It stays hidden wherever the endpoint doesn't answer with JSON (GitHub Pages serves the .php as text; servers without PHP), so the static copies are unaffected. **build.sh** excludes **api/** from the PyPI package's copy of the lab, and **.gitignore** has **ridehail-data/** (a local Apache setup puts the data dir at the repo root). - Tested in the official **php:8.1-cli** container (the host's version): ranks and ties, best-per-name and its displayed spelling, admin list and delete, blocklist, rate limit, and rejections (tampered net/hr, short shift, bad or blocked names, bad market or code, wrong method). The client module was driven in jsdom against it.

Known gaps (Phase F): the game layout on a real phone is untested. Previously: phones can't reach the tab, because the phone tier hides the tab bar (needs the URL deep link). Pressing **g** while on the Game tab hides the tab button but leaves the panel showing. The offer-study citation has no link yet.

2026-09-30: Offer card closer to a real one. Compared with an Uber card and a third-party overlay app's screenshot. The pickup and trip lines are now worded as the platform words them (“6 mins (3.0 km) away”, “10 mins (5.0 km) trip”, with hours as “2 hr 1 min”), with the platform's circle-line-square route marker. “(unpaid)” is gone from the pickup line, as it isn't on real cards; the sidebar legend still says it. The drop-off zone line stays, as our stand-in for the addresses. The Rookie-only “\$0.55 a minute...” line became a boxed right-hand column of \$/km and \$/hr (pickup included), like a third-party overlay app. Pro

still shows no rates. `describe_offer()` now includes `per_km` (the Dollar-a-Km bot uses it). The rate card is still not shown, and there's still no map inset or rider rating.

2026-09-30: Status box shows progress. The status box in the shift pane (Idle / Driving to the pickup / Rider on board) is now tinted in the phase colour instead of only its left border. During a pickup or trip it fills left to right as a progress bar. `GameController.leg_progress()` supplies the fraction (in `frame_payload()` as `leg_progress`): the remaining torus distance to the target, measured against the pickup distance when the leg is first seen, or against the trip's own distance. While idle there's no end to show, so the fill sweeps every 2.4 s (none with reduced motion). The sweep stops while the game is paused or an offer is up, when the world is frozen. The status box is also full width on phones (the phone column had kept the grid's `align-items: start`).

2026-09-30: Pause works with an offer showing. For teaching use, a player can pause (button or space) while an offer is up to study it. Only the card's countdown stops (`OfferCard.pause()/resume()`). The card stays readable, its buttons and keys are disabled, and the timer shows a pause sign. Nothing is sent to the worker: it is already holding its loop for the decision, and a `Pause` message would clear `offerHeldSettings`, so the decision that followed would be dropped. Resuming doesn't move focus to Accept, because the space bar's key-up could then press it. Such a shift sets `shift.pausedOnOffer`: the debrief kicker says "offers paused", and the leaderboard shows the standings but no name form (like ended-early shifts). Pauses between offers gain nothing (no decision is pending), so they don't disqualify a shift. The personal best is unaffected. The flag is set in the browser, so it's honour-level, like the leaderboard's other checks. We decided against a separate untimed "Practice" mode for now; a `#game?...&untimed=1` link switch would be cheap to add later.

2026-09-30: Running costs \$0.56/km, with a source. `ops_cost_per_km` went from \$0.30 (the presets' undocumented `per_km_ops_cost`) to \$0.56, the median cost per km driven in a 2024 report to the City of Toronto (<https://www.toronto.ca/legdocs/mmis/2024/ex/bgrd/backgroundfile-251343.pdf>, p. 26). It includes fixed and variable expenses, mostly variable. (A driver the user knows estimates \$0.46.) An info button (the lab's `app-info-popover`, with game-sized text) next to "Running costs" in the sidebar explains it and links the report. The debrief footnote cites it, and "How it works" says \$0.56. The debrief has the same kind of button after "running costs (... km)" near the top. Its panel adds the shift's own figures (cost, km, cost per hour), and it hangs below the whole line (`.game-info--below`) so it stays on screen wherever the line wraps.

Re-calibration (40 seeds): every net figure falls by about \$7.50-8/hr (0.5 km/min $\times$ 60 $\times$ \$0.26), and the ranking in each market is unchanged, since the cost barely depends on decisions:

Market	Accept-all	Hourly Thinker	Dollar-a- Km	Rate-Card Loyalist	Fleet mean
Busy Friday	\$6.7	\$20.1	\$18.2	\$10.5	\$6.7
Normal	\$7.5	\$8.7	\$9.7	\$8.9	\$7.6
Slow Tuesday	\$1.7	\$0.8	\$1.3	−\$1.9	\$1.1

Slow Tuesday now nets about \$1/hr on average, and a picky player can end a shift in the red. The bots’ thresholds (\$1.00/km, \$0.55/min gross) were not changed.

Leaderboard: OPS_COST_PER_KM is 0.56 in `api/leaderboard.php`, and the new MIN_SCORING_VERSION (‘2026.9.30.2’, the first build with \$0.56) keeps older scores out of the standings (they stay in the database). A submission from an older page is refused (409, “please reload”) before the cost check, so its error makes sense. Raise the constant whenever a game change makes old scores incomparable. Tested in `php:8.1-cli`.

2026-09-30: Pay per hour in three steps. In the debrief’s “Where your time went”, the sentence “per engaged hour ... you grossed \$X; per hour of your whole shift you netted \$Y” changed two things at once (idle time and costs). It is now a three-row table (`hourlyStepsHtml` in `modules/game-debrief.js`): fares per engaged hour → fares per hour of the shift (idle included; earnings ÷ shift hours, worked out in JS) → after running costs (the score). A note says Ontario’s platform-worker minimum is the general minimum wage, \$17.95/hr from 2026-10-01 (ONTARIO_MINIMUM_WAGE), for engaged time and before expenses, i.e. the first row. It links to ontario.ca’s digital platform workers page and the ESA minimum wage page, and says whether the player’s first row was above it (and, if so, whether their net fell below it). Update the constant when Ontario’s minimum wage changes.

2026-09-30: Bots named for their rules. The informal names (Yes-to-Everything, Hourly Thinker, Dollar-a-Km, Rate-Card Loyalist) and their second-line rule text are gone. Each bot is now named for its rule: “Accept every offer”, “Accept only offers of at least \$33 per hour”, “... at least \$1.00 per km”, “... at least the rate card”. The hourly one is stated per hour, not per minute (\$0.55), to match the \$/hr the offer card shows. The names are built from BOT_MIN_PER_KM / BOT_MIN_PER_MIN in `ridehail/game.py`, the thresholds the bots use, so they can’t drift apart. `Bot.rule` and the results’ `rule` field were removed. A note under “How you compare” says pay per km and per hour include the pickup. The rate-card row has an info button (`rateCardInfoHtml`) giving the rate card from `results.params` and saying that offers are the rate card times a random factor. `.game-table-scroll:has(details[open])` lets the panel escape the scroll wrapper while it’s open. Older entries in this log still use the old names. Later the same day the names were shortened to “Takes

every offer”, “Takes \$33/hr or more”, “Takes \$1.00/km or more” and “Takes the rate card or more”. “... or more” says which side of the line is accepted, and “/hr” and “/km” match the offer card’s box. We avoided “threshold” (jargon) and “Minimum” (it could be read as a pay guarantee next to the Ontario minimum wage).

2026-09-30: Leaderboard on the setup screen. Next to the shift code, a preview (`renderBoardPreview` in `modules/game-leaderboard.js`, `#game-setup-board`) shows the top 5 of the board for the chosen market, code and difficulty, and “Top 5 of N drivers”. The player’s remembered name is highlighted; an empty board says “No scores yet...”. It refreshes when the setup screen is shown, when the market changes, 400 ms after the code is edited, and after a link sets the fields. Only the latest request is drawn. It stays hidden where there’s no leaderboard endpoint, and the code field then takes the whole row. On phones the preview sits below the code field.

Part 3: Realistic offer prices (spec, 2026-09-30)

Status: **implemented 2026-09-30** (see 3.11). The answers to the open questions (3.10): Uber only; keep the invented rate card (published rate cards describe rider fares only); keep long-pickup pay (with upfront pricing it’s no longer true that drivers are paid only while a rider is aboard); leave `mean_trip_distance` as it is. Trips stay within the 12 km city.

3.1 Goal and principle

Make the distribution of offers as realistic as possible, while keeping it cheap to compute in the browser. There are two data sources, and each is good for a different thing:

- **Which trips happen** (the trip mix): the Toronto FOIA trip records (`~/src/ridehail-toronto/duckdb/toronto.duckdb`, view `uberx_completed_trips`, 123M+ trips 2023-01 to 2024-10). This is the population. In the game the trip mix comes from the simulation, so we match the simulation to this.
- **What a given trip pays the driver** (the price given the trip): the offer study (`~/src/uberdriver/duckdb/uberdriver.duckdb`, schema `offer`, view `offers_included`, 18,421 offer cards). Its trip mix is biased (too many airport and long trips), but the price of a given trip should be much less biased, provided the features that are over-represented are modelled and then left out of the game.

So: **trip mix from the city’s data, price given the trip from the offer study**. The game gets a small fitted price model (a few coefficients and a residual quantile table), not data.

3.2 Evidence (queried 2026-09-30)

Trip mix:

	Offer study (Uber)	Toronto FOIA (UberX, 2% sample, 1.8M)
Median trip	7.9 km	6.2 km (mean 10.3)
Share over 12 km	38%	28%
Pearson pickups	~12% of all offers (2,157)	~6% (0.1° cell, an upper bound)

Toronto trips of 12 km or less: quartiles 2.6 / 4.3 / 6.9 km, 10–90% 1.7–9.5 km, mean 5.0 km. **The game’s trips are already close to this:** from played Normal shifts, quartiles 3.0 / 4.8 / 7.5 km, 10–90% 1.5–9.5 km, mean 5.2 km. At most a small tweak of `mean_trip_distance` is needed (see 3.5).

Price given the trip (offer study, all platforms, median offer by trip length):

Trip km (me- dian)	1.4	2.9	4.8	6.8	8.9	10.9	13.8	17.9	24.8	37.4
Trip min (me- dian)	5	8	11	14	17	18	20	21	25	35
Median offer	\$4.09	\$5.56	\$7.20	\$8.62	\$10.14	\$11.18	\$13.00	\$15.05	\$19.01	\$25.60
\$/km	3.00	1.96	1.51	1.27	1.15	1.04	0.94	0.84	0.76	0.65

1. **Pay per km falls steeply with trip length.** Our rate card $\times$ a lognormal factor (median 0.92) is too flat. Relative to the real median offer it pays about right on the shortest trips and about 10–20% too much from about 5 km up.
2. **Long pickups are partly paid.** Uber, Toronto pickups, no airport (6,174 offers). Offers were compared within cells of trip km (20 bins) $\times$ trip minutes (10 bins), then averaged by pickup distance:

Pickup km	0.5	0.5–1	1–2	2–3	3–5	5–8	>8
Offer relative to the cell mean	0.95	0.96	1.00	1.03	1.06	1.24	1.33
n	1,106	1,444	1,983	904	539	152	46

This is consistent with a long-pickup payment. The game currently treats every pickup as unpaid.

3. **The spread around the typical price is steady.** A log-linear fit (log fare on log trip km, log trip min, $\log(1 + \text{pickup km})$, airport, platform; all offers) has R^2 0.82 and residual sd 0.265. The sd is 0.23–0.29 in every trip-length band, and the residuals are skewed to the right (10/50/90%: $-0.29 / -0.04 / +0.36$). In that fit, airport pickups pay +24% and Hopp -20% ; Lyft is the same as Uber.
4. Real speed varies with trip length (1.4 km takes 5 min, i.e. 17 km/h; 37 km takes 35 min, i.e. 64 km/h). Game speed is fixed at 30 km/h.

3.3 The price model

`offer = F(trip_km, pickup_km) × _trip, rounded to $0.05`

- **F**, the typical offer, depends on **km only**. Time is left out because game minutes are always $2 \times \text{km}$; in the fit, the real km-to-minutes relation is absorbed into the km terms. Proposed form, to be settled by the fitting script: $\log F = a + b \cdot \log(\text{km}) + c \cdot \log(\text{km})^2 + g(\text{pickup_km})$, with **g** either $d \cdot \log(1 + \text{pickup_km})$ or piecewise linear with a knee near 4–5 km, whichever fits better. **g** is centred so that a typical pickup gives a factor of about 1.
- **_**, the trip’s price luck, is drawn from the **empirical residual distribution**, stored as about 41 quantiles (2.5% to 97.5%) and sampled by inverse CDF with linear interpolation, using the game’s own RNG. This replaces the assumed lognormal and keeps the real right skew. If the spread turns out to depend on trip length, store quantiles for two or three km bands instead.
- **Per trip and per driver.** The pickup distance differs for each driver who is offered a trip, so the offer does too (as on real platforms). Draw once per trip, so the trip’s “luck” is shared, and evaluate **F** for each driver’s own pickup. This changes the bookkeeping in `game.py`:
 - `prices[trip]` becomes the trip’s and rate card, and each offer is computed at dispatch (`describe_offer`).

- Fleet cars take the fast path in `_offer_filter`, but their pay still has to be recorded at dispatch. That’s one evaluation of `F` per fleet dispatch, which is cheap.
- The ledger accrues the **accepted** driver’s offer, keyed by (driver, trip), not by trip.
- “If you decline, the next driver sees the same price” is no longer true, and the design text (1.4) must change.

Fit sample (proposed): offers not excluded, **Uber only** (10,472; its pricing is the one the card imitates, and it avoids platform effects in the residuals), pickup in Toronto, **no Pearson pickups or drop-offs**, trip 15 km (a little beyond 12 so the curve is steady at the edge), **fare** without **bonus** (see 3.9). **Weights**: each offer is weighted by Toronto’s share of trips in its km band (12 km, 1 km bands) divided by the offer study’s share, so the fit concentrates where Toronto’s trips are.

3.4 The rate card

It is kept as a reference: for the rate-card driver (“Takes the rate card or more”), the debrief’s “vs rate card” column and the rate-card info button. It no longer generates offers. With a curved `F`, comparisons with a straight rate card become realistic: short trips often beat it, long trips usually don’t. Option: replace the invented $\$2.50 + \$0.75/\text{km} + \$0.18/\text{min}$ with a documented card. For example, the Toronto effective rate card (`trip_model` in `toronto_opendata.duckdb`) is a rider-fare card, so it would have to be scaled to the driver’s share, or we’d use Uber’s published Toronto driver rates if we can find them. To be decided (3.10).

3.5 Trip mix

Keep GAMMA with the 12 km city (decided). Check the game’s trip lengths against Toronto’s trips of 12 km or less (3.2), and if needed nudge `mean_trip_distance` (now 12 blocks) so the medians match. The game is currently about 0.5 km long at the median. Trips over 12 km (28% of Toronto trips) are left out; the debrief footnote should say so.

3.6 Implementation

1. `utils/fit_offer_model.py`: reads both DuckDB files (paths as options, defaulting to the sibling repos), applies the filters and weights, fits `F` (comparing the forms of `g`) and the residual quantiles, prints diagnostics (3.7a), and **writes** `ridehail/game_offer_model.py`. That is a generated module of constants: the coefficients, the quantile table, and provenance (date, row counts, filters, the offer study’s latest `sync_run` and git commit, the FOIA months used). No data ships in the wheel. The script needs `duckdb`, which is a dev-only dependency (not in Pyodide).

2. `ridehail/game.py`: `price()` / `describe_offer()` use the model (3.3). The ledger is keyed per (driver, trip), and fleet dispatches are priced at dispatch. The `GameParams` multiplier fields go; `price_step` stays.
3. Tests (`test/test_game.py`): replace the lognormal-multiplier test with checks that the median offer by km band and the residual quantiles reproduce the model; offers rise with pickup distance; is shared by the drivers offered one trip; earnings still match completed trips.
4. Re-run `utils/game_calibrate.py` and `utils/game_strategy.py`. Check that the market flip still holds (picky pays when Busy, not when Slow), and re-tune the bots' thresholds (\$33/hr, \$1.00/km) if the new prices make them degenerate.
5. `./build.sh`; raise `MIN_SCORING_VERSION` in `api/leaderboard.php` to the new build.

3.7 Validation

- a. **Fit diagnostics** (printed by the script): residual median and spread by km band and by pickup band (flat means F has the right shape), and the weighted and unweighted fits side by side.
- b. **Game against data**: offers generated by played Normal shifts (the game's own trips and pickups) against the reweighted offer study, trips 12 km. Compare quantiles of offer, \$/km and \$/hr (pickup included), and the median offer by km band. Target: medians within about 5%, 10–90% spreads within about 10%.
- c. **Markets**: the calibration table (as in 2.10) before and after.

3.8 Text and UI knock-ons

- “Unpaid” pickups: the sidebar legend “To a pickup (unpaid)”, the status line “Driving to the pickup (unpaid)”, “How it works” (“The drive to the pickup is unpaid”), and the debrief time split and “Three things”. Pickup *time* is still not paid as such, but long pickups raise the offer. Proposed wording: “not paid by the minute; long pickups raise the offer”.
- The debrief footnote “About the prices” (now “a rate card times a random factor”) describes the new model and its sources, and says trips over 12 km are left out.
- The rate-card info panel (“Each upfront offer is the rate card times a random factor”) needs rewording.
- Design text 1.4 and 1.5 are superseded by this part.

3.9 Left out for now

- **Surge and time of day**. The offers have timestamps (`captured_at`), so a “Busy Friday” premium could be estimated later.
- **Airport trips, trips over 12 km, platform differences** (Hopp –20%).
- **Speed varying with trip length** (fixed 30 km/h).

- **bonus** (shown on some cards): to check how often it's non-zero and whether it should be added to the fare.
- **Selection in the offer study.** The capture tool is assumed to have saved offers regardless of price; if it saved some offers selectively, the price model inherits that.

3.10 Open questions

1. Uber only, or Uber + Lyft pooled (a larger sample, but Lyft's pricing differs in shape)?
2. Keep the invented rate card, or move to a documented one (3.4)?
3. Should long-pickup pay be in the model now? It's realistic, but it changes the game's "pickups are unpaid" lesson (3.8).
4. Nudge `mean_trip_distance` to match Toronto's median, or leave the game's trip mix as it is (0.5 km long at the median)?

3.11 Progress log

2026-09-30: implemented.

- `utils/fit_offer_model.py` (`uv run --with duckdb --with pandas python utils/fit_offer_model.py [--validate N]`) fits the model and writes `ridehail/game_offer_model.py`, a generated module of constants with provenance. The fit sample is 4,945 Uber offers: Toronto pickup, no Pearson, trips 15 km. Each is weighted by Toronto's share of trips in its 1 km band (a 2% sample of 2023-01 to 2024-10, 1.4M trips 15 km), over the study's share. The Toronto sample is seeded, so refits are repeatable.
- The chosen form is $\log F = 1.3995 + 0.3777 L + 0.0451 L^2 + 0.0132 p + 0.0187 \max(0, p - 4)$, with $L = \log \text{trip km}$ and $p = \text{pickup km}$. The piecewise-linear pickup term just beats $\log(1 + p)$; knees at 3–6 km are indistinguishable. An 8 km pickup raises F by about 20%. The weighted residual sd is 0.31, wider than the 0.265 of the fit with minutes, because speed variation is now part of the luck. The residual medians are flat (within ± 0.06) across trip and pickup bands. The luck tables (99 quantiles, bands <2, 2–6, >6 km) have 10/50/90% about $-0.36 / -0.04 / +0.40$. The minimum offer is \$3.06.
- `ridehail/game.py`: `price(trip)` is now `(rate_card, luck)`, drawn once per trip. `offer_price(trip, dispatch_distance)` is $F \times e^{\text{luck}}$ for that driver's pickup, rounded to \$0.05. `agreed[(driver, trip)]` records the price at dispatch for every taker (the fleet's fast path, bots and the player), and earnings accrue from it. The `GameParams` multiplier fields were removed. The results' `params` gain `offer_study_offers`.
- Validation (20 Normal shifts, 18,397 game offers, against the reweighted study 12 km):

- The median offer by 2 km band is within 4% in every band.
- The offer quantiles 25–90% are within 3%; the 10th percentile is 6% low (\$4.10 against \$4.37).
- \$/km (pickup included) is about 12% low, because game pickups are longer (median 2.0 km against 1.2).
- \$/hr is higher (median \$35 against \$30), because game cars do 30 km/h, faster than real short trips.
- Markets (`game_calibrate.py`, 40 seeds). Earnings rose (fleet mean: Busy \$6.7 → \$11.3, Normal \$7.6 → \$10.6, Slow \$1.1 → \$3.2), because offers for typical trips pay more than the old model did. The sweep (`game_strategy.py --skip-refine`, 40 codes, same seat per code) confirms the flip still holds:

Market	Best \$/min threshold	Net at best	Takes every offer
Busy Friday	\$0.60 (\$36/hr)	\$19.5	\$10.8
Normal	\$0.50 (\$30/hr)	\$12.8	\$10.9
Slow Tuesday	none (accept all)	\$3.3	\$3.3

The bot comparison in `game_calibrate.py` has the \$33/hr bot ahead of accept-all in Slow (\$5.4 against \$3.7). The bots sit in different seats, so that is within the seat-to-seat noise; the same-seat sweep is the one to trust. The bots’ thresholds are unchanged.

- Wording: “(unpaid)”/“(paid)” was removed from the status box and the legend. “How it works”, the hourly-steps note and the pickup insight now say an offer is a fixed price for pickup and trip together, a long pickup raises it a little, and idle time is never paid. The price footnote describes the model (Uber only, 4,945 cards, weighted to Toronto’s trip lengths, no airport, no trips over 12 km, rate card for comparison only). The rate-card info panel says offers aren’t set from it.
- The leaderboard’s `MIN_SCORING_VERSION` is 2026.9.30.5.
- Tests: the lognormal test was replaced by tests of the luck quantiles, the shape of F (per km falls, rises with pickup) and shared luck across drivers (16 game tests).

Part 4: What riders paid (2026-09-30)

Status: **implemented 2026-09-30**. Busy Friday fares (4.6) are written up but not implemented.

4.1 Goal

Say what riders probably paid for the trips a driver took, and how much of it reached the driver, using City of Toronto data limited to 2026.

4.2 Source

`~/src/ridehail-toronto/duckdb/toronto_opendata.duckdb`, table `trips`: the City's PTC open data (<https://open.toronto.ca/dataset/private-transportation-companies-summary-and-trip-data/>). The rows are hourly origin-destination cells (ward to ward) with the mean fare, distance and duration of `trips_total` trips, from 2018 to 2026-07. The fare is trip fare + City of Toronto fees + HST, excluding tips and promotional discounts, for all platforms and products together (no breakdown). The offer study's Uber offers date from 2026-01 to 2026-08 (median March), so the two sources line up in time.

4.3 The fare model

$\text{fare} = \text{base} + \text{per_km} \times \text{km}$, fitted by weighted least squares (weight = trips) on 2026 cells, Toronto to Toronto, averaging 15 km: **\$8.11 + \$1.018/km** (2026-01 to 2026-07, 40.4M trips, cell rse \$3.39). The fixed \$8 absorbs the booking fee, minimum fares and City fees. Distance only: game minutes are always $2 \times \text{km}$, and a model that is linear in km is unbiased on cell means if fares are linear within a cell. Adding duration fits cells better ($\$4.42 + \$0.25/\text{km} + \$0.57/\text{min}$), but it needs realistic minutes, which the game doesn't have. Month to month, the fitted 5 km fare ranged from \$12.35 (March) to \$14.16 (January). The project's own check against the FOIA UberX-only data (2023-24) found the blended typical fares within 1–4% of UberX, so the product mix is a small effect.

4.4 Per-trip fares share the trip's luck

A first version used the typical fare for every trip. Picky strategies then showed driver shares over 100% (the \$1.00/km bot: 123%), because they select trips whose offers came out high while the rider fare stayed typical. On real trips the two move together: the uberdriver pay-statement analysis (`~/src/uberdriver/analysis_upfront_pay.md`, US data) found the platform's take at $16\% \pm 7\%$ per trip, far steadier than offers (luck sd 0.31). So each trip's rider fare is the typical fare $\times e^{\text{luck}} / \text{mean}(e^{\text{luck}} \text{ in its band})$. The trip's luck is the one its offer uses, and the normalisation keeps the average fare equal to the City's. The driver's share then varies with trip length and pickup, not with luck.

4.5 Implementation

- `utils/fit_offer_model.py` also fits the fare (`fit_rider_fare`, `--opendata-db`) and writes `RIDER_FARE_BASE`, `RIDER_FARE_PER_KM`, `RIDER_FARE_MONTHS` and `RIDER_FARE_TRIPS` into `ridehail/game_offer_model.py`.
- `ridehail/game.py`:
 - `typical_rider_fare(km)`, `rider_fare(trip)` (4.4) and `HST_RATE = 0.13`.
 - Every offer dict has `rider_fare`.
 - `Ledger.rider_fares` accrues per block alongside earnings (with `accrued_fare`), so a trip in progress at the end of the shift counts the same share of both.
 - The summaries add `rider_fares`, `rider_hst` and `driver_share` (earnings $\div$ pre-HST rider fares).
 - `results.params` adds the fare model and months.
- Debrief (`modules/game-debrief.js`):
 - a “Share of fares” column in “How you compare”, with a note;
 - a “What your riders paid” section after the hourly steps (total, HST, your earnings, your share, “the rest went to the platform, apart from the City’s per-trip fees”);
 - a “Rider paid[†]” column in “Your offers”;
 - “About the prices” cites the open data for the fare estimate, with the months, and explains 4.4.
- The offer card doesn’t show the rider fare: drivers don’t see it when deciding.
- Scoring is unchanged, so `MIN_SCORING_VERSION` stays. The leaderboard ignores the new fields.
- Tests: rider fares match completed trips plus the part in progress; the typical fare is linear; the average fare over many trips equals the typical fare; the per-trip driver share varies $< 5\%$ between its 10th and 90th percentiles (18 game tests).

Results: fleet drivers receive 74% (Normal, Slow) to 78% (Busy, longer paid pickups) of what riders paid before HST. In one Busy shift, the strategies ranged from 71% to 82%.

4.6 Later: Busy Friday fares (not implemented)

Rider fares vary with the time of day and the day of the week, and the open data has hourly cells. Fit the fare model (or just its level) for Friday evenings, e.g. 18:00–02:00, and use it for the Busy market, and a quiet weekday slot for Slow. The offer study’s timestamps (`captured_at`) could give the same by-time scaling for driver offers, so fares and offers would move together. The market descriptions on the setup screen would then describe a real time of week, not just the balance of supply and demand.

Part 5: How realistic are the markets? Diagnosis (2026-09-30)

Status: **diagnosis done**. Option (a) was tried on 2026-09-30 and reverted (5.6): the markets are unchanged (80 / 96 / 130 cars, 5 requests/min, inhomogeneity 0.5). Still open: the simulation-core fix (5.7), the 30 km/h speed (5.8) and the comparison with the City-commissioned report (5.9).

5.1 The question

Players net about \$11–12/hr (Busy, Normal), which seemed high next to the City-commissioned report. Is idle time underestimated?

5.2 How the markets were set

MARKET_SHARED sets city 24, 5 requests/min, mean trip 12 blocks (GAMMA), inhomogeneity 0.5 and a 1-minute pickup stop. The markets differ only in fleet size: Busy 80, Normal 96, Slow 130. The fleet sizes were tuned with `game_calibrate.py` to game-design targets (P1 0 / 0.2 / 0.5 and the strategy flip), never to real time shares.

5.3 The City’s 2026 time split

From `operating_hours` (hourly, per vehicle, 2026-01 to 2026-07, Toronto time):

- Idle: from 11% (weekday 8 am) to 46% (Tuesday 1 am); 30% on average.
- Pickup (en route plus waiting): 13–16% at nearly every hour, of which waiting at the pickup is about 3%.
- With a rider: 43–56%.
- Slices: Fri/Sat 19–01h 0.31 / 0.13 / 0.56; weekday 10–15h 0.33 / 0.13 / 0.54; Tue/Wed 01–05h 0.40 / 0.16 / 0.43.

The game has Busy 0.00 / 0.50 / 0.50, Normal 0.19 / 0.30 / 0.51 and Slow 0.46 / 0.16 / 0.38. Busy is unrealistic, and Normal’s pickups are twice the City’s. The City’s unpaid share (idle plus pickup, about 45%) is close to the game’s Normal (49%), but it’s mostly idle time rather than pickups. The price side agrees with the offer study: net per engaged hour (engaged costs only) is \$16.70 in the game’s Normal, against the workbook’s \$15.60.

5.4 Diagnostic (`utils/game_pickup_diagnostic.py`)

City 24, Normal (96 cars, 5 requests/min) unless noted; 3 seeds, 180 measured blocks.

Case	P1 / P2 / P3	Pickup (blocks) vs ideal	Core: requests / idle cars
inh 0.5 (now)	0.19 / 0.30 / 0.51	4.8 vs 3.7	0.63 / 0.05
inh 0	0.27 / 0.22 / 0.52	3.2 vs 3.0	0.25 / 0.25
inh 0.4	0.22 / 0.26 / 0.51	4.1 vs 3.4	0.54 / 0.07
inh 0.75	0.06 / 0.43 / 0.51	7.6 vs 5.3	0.82 / 0.03
inh 0.5, no pickup stop	0.27 / 0.21 / 0.51	4.1 vs 3.0	
2× density (192/10), inh 0	0.33 / 0.16 / 0.51	2.1 vs 1.9	
2× density, inh 0.2	0.31 / 0.17 / 0.51		0.11 idle in core
2× density, inh 0.25	0.30 / 0.18 / 0.51		0.08 idle in core
2× density, inh 0.4	0.28 / 0.21 / 0.51	3.0 vs 2.0	0.54 / 0.05

Findings:

1. **Cars drain out of the core.** Inhomogeneity concentrates *origins* only. With GAMMA (and EXPONENTIAL, RAYLEIGH) trip distances, `Trip.set_destination_sampled` places the destination at the drawn distance from the origin, in a random direction. So `inhomogeneous_destinations` has **no effect** for these distributions; only UNIFORM uses it, via `City.set_location`. Trips leave the core, idle cars collect in the outskirts (5% of idle cars are in the core, which is 25% of the area, against 63% of requests), and pickups come in from outside. Idle cars drift at random (`idle_vehicles_moving`) rather than returning towards demand. This is what makes P2/P1 depend so strongly on inhomogeneity. Also, `RideHailSimulation.__init__` sets `mean_trip_distance = city_size // 2` whenever `inhomogeneous_destinations` is on, whatever the distribution.
2. **Density.** At the same inhomogeneity, doubling cars and requests shortens pickups (ideal distance $1/\sqrt{\text{idle density}}$).
3. **The pickup stop** (`pickup_time = 1`) is about 5% of all time, against the City's 3% waiting.
4. **City 48** at the same density ($\times 4$ cars and requests) overloads: P1 0–4%, pickups 9–13 blocks, and 8–17% of riders give up. Trips are longer there because the GAMMA draw is cut off at the city size (24 or 48 blocks), so the 24-block city loses more of the long trips (mean about 10.4 against about 12 blocks). The 24-block city's match to Toronto's trips 12 km (Part 3) partly depends on this cut-off. With 520 cars ($\times 1.35$), city 48 has P1 0.36, P2 0.21, P3 0.44 (inh 0.5). At 2× density (768/40), inh 0 gives 0.21 / 0.19 / 0.60; inh 0.4 gives 0.06 / 0.38 / 0.57.

5.5 Options for step 2 (to decide)

- a. **Keep city 24; double the density and lower inhomogeneity to about 0.2–0.25.** This reproduces the City’s average split (0.30 / 0.17 / 0.51) with no core change. Busy and Slow would then be set as fleet-size variations around it, targeting the City’s busy-hour and quiet-hour idle shares (about 15–20% and 40–45%), both with pickups near 15%. The web map has twice as many cars.
- b. **Make destinations follow inhomogeneity for the sampled distributions** (a core change, e.g. rejection-sampling destinations towards the core at the drawn distance). This keeps a stronger downtown, and it would also fix the silent no-op for other users of those distributions. It needs care with the torus, and regression tests for existing configs.
- c. **Move to city 48.** This needs `mean_trip_distance` re-checked against Toronto’s trip mix, since the cut-off changes, and a fleet re-tuned for the longer trips. It’s four times the cars to animate.
- d. **Shorten the pickup stop** (0 blocks, or make it part of the block). This affects the animation timing, so it’s small but not free.

Whatever is chosen, re-run `game_calibrate.py` and `game_strategy.py`: the “picky pays when Busy” result depends on Busy’s zero idle time and long pickups.

5.6 Option (a): tried, then reverted

MARKET_SHARED: 10 requests/min (was 5) and inhomogeneity 0.25 (was 0.5); city 24, trips as before. The fleet sizes come from a search (4 seeds per size), targeting the City’s 2026 split:

Market	Cars (was)	Idle / pickup / with rider	City reference
Busy	175 (80)	0.20 / 0.24 / 0.56	busier hours (11–25% idle)
Normal	192 (96)	0.30 / 0.18 / 0.51	2026 average 0.30 / 0.15 / 0.55
Slow	225 (130)	0.43 / 0.14 / 0.43	Tue/Wed 01–05h 0.40 / 0.16 / 0.43

The overload edge is sharp: 160 cars give 0.02 / 0.43 / 0.56 with 6% of riders giving up, and 165 cars give 0.10 idle. So Busy stays at 175. Nobody gives up at the three market sizes.

Strategy sweep (`game_strategy.py --skip-refine`, 40 codes, same seat, net \$/hr):

Market	Takes every offer	Best threshold	Net at best
Busy	\$13.51	\$0.50/min (\$30/hr), accepts 77%	\$15.07 (+\$1.56)
Normal	\$11.15	\$0.60/min, accepts 65%	\$11.24 (flat)
Slow	\$5.15	\$0.40/min, accepts 95%	\$5.49 (flat)

Fleet mean: Busy \$13.20, Normal \$10.35, Slow \$6.19. **The game’s lesson changes.** With realistic idle and pickup times, being picky gains a little only when it’s busy, and nothing in normal or slow markets. High thresholds cost money everywhere. The old “picky roughly doubles earnings when Busy” came from the unrealistic Busy market (no idle time, pickups half of all time). Earnings levels changed little (Normal \$10.6 $\rightarrow$ \$10.35), so the gap to the City-commissioned report isn’t explained by idle time. It lies in the price side (offers, and the game’s steady 30 km/h making paid minutes productive) or in the report’s definitions (still to compare).

Decision (2026-09-30): reverted. Earnings hardly moved, and with realistic markets thresholds barely help, which removes most of the strategy in the game. The user reverted the uncommitted changes: the market values in `ridehail/game.py`, the setup screen’s descriptions, the leaderboard’s `MIN_SCORING_VERSION` (back to 2026.9.30.5) and the version bump. These notes and `utils/game_pickup_diagnostic.py` were kept. The tuned values are here in case they’re wanted later (for example, a “realistic” market alongside the three game markets). Worth knowing: the game’s markets are deliberately not realistic in idle and pickup time; Busy in particular has no idle time and half of all time spent on pickups. If this is ever revisited, the names “Busy Friday” and “Slow Tuesday” fit loosely. In the City’s data Friday evenings are about 23–35% idle, and Busy matches weekday rush hours and weekend afternoons better. See also the Busy Friday fares idea (4.6).

5.7 Outstanding: destinations ignore inhomogeneity (simulation core)

With GAMMA, EXPONENTIAL or RAYLEIGH trip distances, `Trip._set_destination_sampled` (`ridehail/atom.py`) places a destination at the drawn distance from its origin, in a random direction. `inhomogeneous_destinations` is silently ignored; only UNIFORM honours it, via `City.set_location(is_destination=True)`. In addition, `RideHailSimulation.__init__` resets `mean_trip_distance` to `city_size // 2` whenever `inhomogeneous_destinations` is on, whatever the distribution. The effect: inhomogeneity concentrates origins only, trips carry cars out of the core, and idle cars collect in the outskirts (5% of idle cars downtown against 63% of requests at inh 0.5). This inflates pickups whenever inhomogeneity is high. A fix would draw destinations towards the core at the sampled distance, for example by rejection sampling among the candidates at that distance, weighted by core membership. It would need regression tests for existing configs, and the game’s markets would need re-tuning afterwards (inhomogeneity could then go back up).

5.8 To investigate: the fixed 30 km/h speed

The game runs at one block (0.5 km) per minute, 30 km/h, for every trip and pickup. Real Toronto trips are much slower, and their speed depends on length. From 2026 open-data cells (Toronto to Toronto, trip-weighted; cell means, so approximate):

Trip km (cell mean)	1.9	2.5	3.5	4.4	5.5	6.5	7.5	8.4	9.5	10.5	11.5
Minute	5.9	9.6	11.4	12.3	13.8	15.6	17.7	18.7	20.2	22.7	23.5
km/h	14	16	18	22	24	25	25	27	28	28	29

Most trips are 2.5–5.5 km, at 16–24 km/h, so a typical game trip takes only about 55–80% of its real time. Offers are priced by km (Part 3), so the game pays a real trip’s money for fewer minutes. Paid time is too productive, and \$/hr is inflated (Part 3’s validation: median \$/hr \$35 in the game against \$30 in the offer data). Pickups are also driven at 30 km/h, which shortens unpaid time too. This is the most likely reason the game’s earnings sit above the City-commissioned report’s (5.6 shows idle time isn’t it).

Ideas, none tried yet: - **A slower block.** Make one block about 0.35–0.4 km at one minute (21–24 km/h). Then the city is 8.4–9.6 km across, trips measured in blocks are shorter in km, and `F(km)` pays less per minute. This needs `km_per_block`, the offer model’s km inputs, the running cost per km, the debrief’s “12 km” wording and the trip-length match (Part 3.5: re-check `mean_trip_distance` against Toronto’s km mix) to change together. - **Price offers by minutes as well as km.** Refit `F` with the real trip minutes (the offer study has `trip_min`) and feed it realistic minutes for the game’s km (from the table above), not the game’s own. This changes pay without changing the map, but the card’s minutes would still be the game’s. - **Length-dependent speed.** Short trips slow, long trips fast. The simulation moves every car one block per block, so this would be a core change. Probably not worth it.

Check any change with `fit_offer_model.py --validate` (\$/hr against the data), `game_calibrate.py` (earnings levels) and `game_strategy.py` (does the strategy lesson survive?).

5.9 To do: compare with the City-commissioned report

The user noted that the report’s driver earnings are lower than both the game’s (about \$11–12/hr net in Busy and Normal) and the offer study’s (\$15.60/hr after costs, idle time excluded). We still need the report’s figure and its definition (per logged-in or engaged hour; which expenses) to compare like with like.

5.10 Is draining the core realistic? Tidal flows in Toronto

The user asked whether the drain reflects a real effect: mornings bring trips into the centre, and evenings take them out. From 2026 open data (**trips**, the core taken as wards 10 Spadina–Fort York, 11 University–Rosedale and 13 Toronto Centre; local time):

	Start downtown	End downtown	Net into the core
Weekday, all day	29.0%	27.5%	−1.5 points
Weekend, all day	33.1%	30.0%	−3.1 points

Net by hour, in points (end share minus start share):

- Weekday: 06–10h +3 to +5 (inbound); 11–18h 0 to ± 2 ; 19–23h −3 to −10; 00–04h −11 to −15.
- Weekend: daytime about 0; 22–23h −3 to −6; 00–04h −11 to −23 (the peak is 03h).

The game (inhomogeneity 0.5, GAMMA distances; the core is 25% of the area) starts 63% of trips in the core and ends 31% there: **−32 points at every hour**. At inhomogeneity 0.25 it’s 44% / 28%, −16 points.

So there is a real tidal effect: a mild inflow in the weekday morning, and an outflow late at night, strongest when the bars close at weekends. But the game’s drain is stronger than Toronto’s most extreme hour and runs all day. It also can’t be separated from how busy downtown is. In Toronto, downtown attracts trips about as strongly as it generates them (29% / 27.5%, from a small share of the city’s area), and the model can’t produce that with sampled trip distances (5.7). Real pickups also stay at 13–16% of time even in the hours with the strongest outflow, presumably because drivers reposition towards demand, while idle cars in the model drift at random.

Implication for the 5.7 fix: give origins and destinations **separate concentration settings** rather than one switch. Equal settings would give a busy, balanced downtown (daytime); a little more on origins would give the evening outflow; more on destinations the morning inflow. Markets could then represent real times of day (compare 4.6), with a drain of realistic size.

5.8, continued (2026-09-30): variable speed, and slowing idle cars.

- **“Time = distance” carries weight in the core model, beyond display.** City-scale pricing folds per-km and per-minute prices into one per-block price (**simulation.py**, around 533 and 1063). **WAIT_FRACTION** equilibration uses **TRIP_DISTANCE** as ride time (around 1336). The presets’ Little’s-law relation $P3 = \text{demand} \times \text{trip length} / \text{vehicles}$ assumes one block moved per block. Moving a P2/P3 car with a probability < 1 would be trivial in itself (as idle cars already do), but every consumer

above would have to separate time from distance. Decision: keep uniform speed in the general model. If ever wanted, the most natural form is a **congested core** (a lower move probability downtown), which gives length-dependent average speeds without a speed-by-length rule.

- **Slowing idle cars is already supported:** `idle_vehicles_moving` is a probability (`Vehicle.update_location`); the game sets 1.0. City 2026 (`summary_stats`, distance share $\div$ time share) gives speeds relative to on-trip of: idle 0.45; en route 0.61 (0.48 including waiting). Game shifts, accept-all fleet, 8 codes, idle moving 1.0 $\rightarrow$ 0.45:

Market	Idle / pickup / rider	Net \$/hr	km/hr
Busy	unchanged (no idle)	\$10.79 $\rightarrow$ \$10.85	28.5 $\rightarrow$ 28.5
Normal	0.20/0.30/0.50 $\rightarrow$ 0.19/0.31/0.51	\$10.85 $\rightarrow$ \$12.64	28.4 $\rightarrow$ 25.3
Slow	0.48/0.16/0.36 $\rightarrow$ 0.45/0.18/0.37	\$3.14 $\rightarrow$ \$7.44	28.8 $\rightarrow$ 21.4

It barely changes the time split, and it raises earnings through lower idle running costs, away from the City-commissioned report. It's realistic for costs, and it makes declining cheaper in slow markets, so the strategy results would need re-checking.

- **Recommended experiment: a slower block**, with no change to the model. One block = one minute = about 0.37 km (22 km/h, typical of Toronto trips under 12 km), via `mean_vehicle_speed / minutes_per_block` and the game's `km_per_block`. Knock-ons in the game layer: offers (F by km), running costs per km, the city's size in km (24 blocks = 8.9 km; about 32 blocks for 12 km trips, which needs more cars; see 5.4), `mean_trip_distance` to keep Toronto's km mix, the debrief's "12 km" text, then `fit_offer_model.py --validate`, `game_calibrate.py` and `game_strategy.py`. Probably combine it with idle moving at 0.45. Judge it as in 5.6: do earnings come down, and does the strategy lesson survive?

5.8 experiment (2026-09-30): the slower block, implemented on dev.

The model is unchanged: a block is still a minute and a distance. Only the game's scale changed.

- `ridehail/game.py`:
 - `KM_PER_BLOCK` = 0.37 (22 km/h), used by `GameParams.km_per_block` and `mean_vehicle_speed`.
 - `MARKET_SHARED`: city 32 (11.8 km, so "12 km" still holds), `mean_trip_distance` 16 blocks, demand 9/min ($5 \times$ area ratio), inhomogeneity 0.5.

- **MARKETS:** Busy 180, Normal 215, Slow 300.
- Trip lengths: 10/25/50/75/90% 1.5 / 2.6 / 4.4 / 7.0 / 9.2 km, mean 4.9 (Toronto 12 km: 1.7 / 2.6 / 4.3 / 6.9 / 9.5, mean 5.0).
- The fleets were searched (3 seeds) to keep the earlier split. Busy 0.00 / 0.46 / 0.54 with 15% of riders giving up; Normal 0.19 / 0.26 / 0.54; Slow 0.47 / 0.14 / 0.39.
- Other files: `game-tab.js` `GAME_CITY_SIZE = 32`; `leaderboard.php` `KM_PER_MINUTE = 0.37` and `MIN_SCORING_VERSION 2026.9.30.10`; the sidebar’s idle cost “about \$12 an hour” (was \$17); the debrief footnote gives the 22 km/h; `fit_offer_model.py --validate` uses the game’s minutes per km; tests use `km_per_block` rather than 0.5.
- Offer validation (20 Normal shifts): the median offer by km band is within about 5%. **\$/hr (pickup included): median \$27.21 in the game against \$28.45 in the data** (before: \$35 against \$30). The spread is wider, because all trips run at 22 km/h.
- Calibration (40 seeds), fleet mean net \$/hr: Busy \$9.29 (was \$11.25), Normal \$9.14 (was \$10.64), Slow \$2.75 (was \$3.16), about 15% lower. Offers per shift for an accept-all player: 8.1 / 8.1 / 5.6. Slow now averages 25 idle minutes per offer (was about 12).
- Same-seat sweep (40 codes): Busy \$9.04 → \$17.42 at \$0.50/min (+\$8.40, accepts 20%); Normal \$10.09 → \$11.39 at \$0.50/min (+\$1.30); Slow \$4.12 → \$4.53 at \$0.40/min (flat, within noise). **The lesson survives**, as before (Busy +\$8.70, Normal +\$1.90). Net pay rises and then falls with the threshold in every market: a moderate threshold (declining the worst offers) pays when offers come quickly, and very high thresholds (declining nearly everything, e.g. Busy \$0.90/min, 2% accepted, −\$0.79/hr) lose everywhere, because idle time and its running costs outweigh the better trips.
- To check in the browser: the 32-block map with 180–300 cars (smoothness, worker frame time), and the longer idle waits in Slow. `WARP_IDLE` in `game-tab.js` could be lowered if they drag.

Part 6: Rate helper vs Platform only (2026-10-01)

6.1 Decision

The dormant “Pro” difficulty is replaced by a visible choice of **offer screen**, which differs from the default in one thing only: whether the offer card carries the **rate helper**, the boxed \$/km and \$/hr (pickup included, over the whole offer). In the real world that box comes only from third-party apps such as GigU; a platform’s own card doesn’t have it. So the choice is about information, not skill, and “Rookie/Pro” (which suggested a skill ladder, and made the helper look like training wheels) is gone.

- Names: **Rate helper** (default) and **Platform only**. Internal key `card`, values `helper` / `platform` (`ridehail.game.CARDS`).
- Both have the 8 s timer and no acceptance-rate rule. Pro’s 5 s timer and

acceptance rule are retired; the rule stays in `GameParams` as a dormant option (1.8).

- The default stays Rate helper, so the existing scores (all made with it) stay on the default board.

6.2 Setup screen

After the market choice, a fieldset whose legend is the user's wording: "Third-party apps help drivers by showing the pay rate. Use a helper app?" with two options: - *Rate helper*: "Each offer also shows what it pays per km and per hour, pickup included." - *Platform only*: "Just the platform's card: the price, the pickup and the trip."

The last choice is remembered in `localStorage(ridehail.game.card)`; a link's `card=` overrides it. "How it works" adds to the offer bullet that the rate helper shows pay per km and per hour, and that each choice has its own leaderboard.

6.3 Offer card

With the rate helper, the box is styled as an add-on laid over the platform's card, not part of it: a dashed border, a neutral background and a "Rate helper" caption. With Platform only the box is hidden and the route column takes the full width.

6.4 Leaderboards and labels

- **Separate boards**: a board is shift code $\times$ market $\times$ offer screen. Mixing would rank Platform-only players against players with more information. The setup preview follows the selected offer screen, and both board titles name it ("Normal · Platform only").
- The debrief kicker and share line always name the offer screen (the old Rookie label was empty). Shift links always carry `card=`, so a friend plays on the same board.
- Personal bests are keyed by market + card; a Rate-helper best falls back to the old `...<market>.rookie` key.

6.5 Debrief

- Platform only: a note above "Your offers" says the "Per hour*" column is what a rate helper app would have shown on each offer, so the information the player went without is shown afterwards.
- Platform only: a note under "How you compare" says the automated drivers that go by \$/km or \$/hr see what a rate helper would show (two of the four bots decide on those figures).

6.6 Implementation

- `ridehail/game.py`: `DIFFICULTIES` → `CARDS`; `create_game(..., card=)` validates it. `test/test_game.py`: `test_offer_screens`, and the acceptance-rule test now turns the rule on through `GameParams`.
- `docs/lab/worker.py`: reads `card` from the game settings.
- `docs/lab/game-tab.js`: `CARDS`, remembered choice, links (`card=`; old `difficulty=rookie|pro` links map to `helper|platform`).
- `modules/game-offer.js` (`show(offer, seconds, helper)`), `modules/game-debrief.js`, `modules/game-leaderboard.js`, `components/game-tab.html`, `style.css`.
- `api/leaderboard.php`: column `difficulty` → `card`, `CARDS`. On the first request after the update, `migrate_difficulty()` deletes the old Pro rows (their rules aren't comparable), sets the Rookie rows to `helper`, and re-names the column. A request without `card` from a page loaded before the update counts as `helper`, unless it says `difficulty=pro`, which gets the 409 “please reload”. Tested in `php:8.1-cli`: migration, board queries for both cards, legacy parameters. `MIN_SCORING_VERSION` is unchanged (Rate-helper scoring didn't change).
- Version 2026.10.1.0 (wheel rebuilt). The web needs the new wheel: the old `create_game()` doesn't accept `card`.

6.7 During the shift (2026-10-01)

Above the sidebar's “Shift time left”, a line names the market and offer screen being played, e.g. “Busy Friday · Rate helper” (`#game-shift-label`, set in `GameTab.start()`). It has its own line because the 300px sidebar can't fit it beside the clock label.

6.8 Your car's ring (2026-10-01)

On the bigger map the player's thin dark ring (2.5px, $1.7\times$ the car's radius) was hard to spot. It is now violet `#7c3aed` (a colour nothing else on the map uses), 4px wide at $2\times$ the radius, with an 8px white halo underneath (`game-map-overlay.js` `RING_COLOR`, `--game-you` for the legend swatch). “How it works” says “violet ring”. The user found it an improvement.

Pulse (kept, 2026-10-01). `pulseGameCar()` in `game-map-overlay.js` plays a “here you are” pulse: 3 violet rings, 250 ms apart, each growing from the ring's size to $3\times$ and fading over 1 s (1.5 s in all), drawn under the ring. `GameTab.onFrame()` calls it on the shift's first frame and when an offer appears. While it plays, a `requestAnimationFrame` loop calls `chart.draw()`; it does nothing with reduced motion or without a map, and `setGameOverlay(null)` (game stopped) cancels it.

6.9 To check in the browser

The setup fieldset (desktop and phone), the dashed helper box and the full-width Platform-only card (desktop, tablet bottom sheet, phone), both board

previews, and the debrief notes for a Platform-only shift.